One Experience from Yesterday

O • Objective	What happened?
R • Reflective	How did you respond?
I • Interpretive	What did you learn from it?
D • Decisional	What will you do today?

Choose how much personal feeling to share. Listen until your partner finishes. Work without AI today.

© 2026 Inspire | Confidential

Everyone Has a Turn

About one minute each, with brief feedback from your coach.

- Name one real event. Point to what you saw or did.
- Connect your learning to that experience.
- Choose a first action you can take today.

What support will help you take that action?

© 2026 Inspire | Confidential

How Did the Example Become Clear?

Selected learners show their submitted Day 1 Homework and the original conversation.

1. What was unclear in the request?
2. What question did you ask?
3. Which answer supports the expected result?

Learn from the submitted and graded version. The submission and score stay fixed.

© 2026 Inspire | Confidential

A Clear Example and Correct Code

A specific question helped us confirm an expected result.

Once an example is clear, what evidence shows that the code meets it?

Keep the original conversation. Use the actual work to explain the clarification process.

© 2026 Inspire | Confidential

INSPIRE AI

Engineering Practices Then and Now

Day 1: AI says it is done.
What makes you accept the result?

Discount • Free shipping • A question, an example, a check



© 2026 Inspire | Confidential

What made you accept a result?

Think of the discount demo
or your free-shipping exercise.

Name one question, one confirmed example,
or one result you actually checked.

Think for 15 seconds. Then share your evidence.

© 2026 Inspire | Confidential

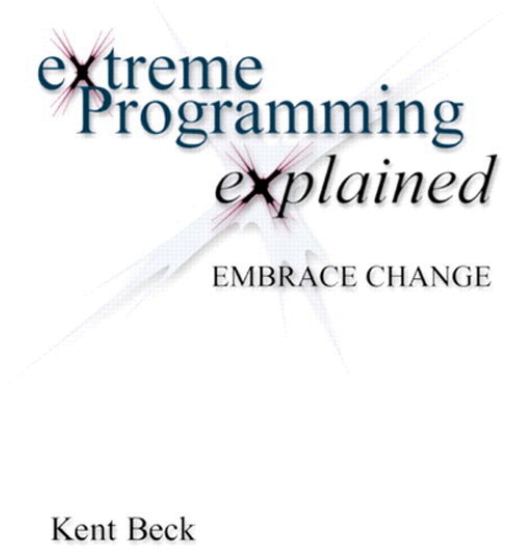
Engineering practices, then and now

1999	2008	2025	2026
Extreme Programming	**Clean Code**	**Augmented coding**	**Agent workflows**
Kent Beck	Robert C. Martin	Kent Beck	Willison • Böckeler • Martin
Practices that support each other	Read, judge and improve code	Delegate more implementation	Different ways to obtain feedback

When the program changes, what evidence lets us keep trusting it?

© 2026 Inspire | Confidential

Kent Beck



First edition • 1999
Publisher cover: Addison-Wesley / InformIT

Published 5 October 1999

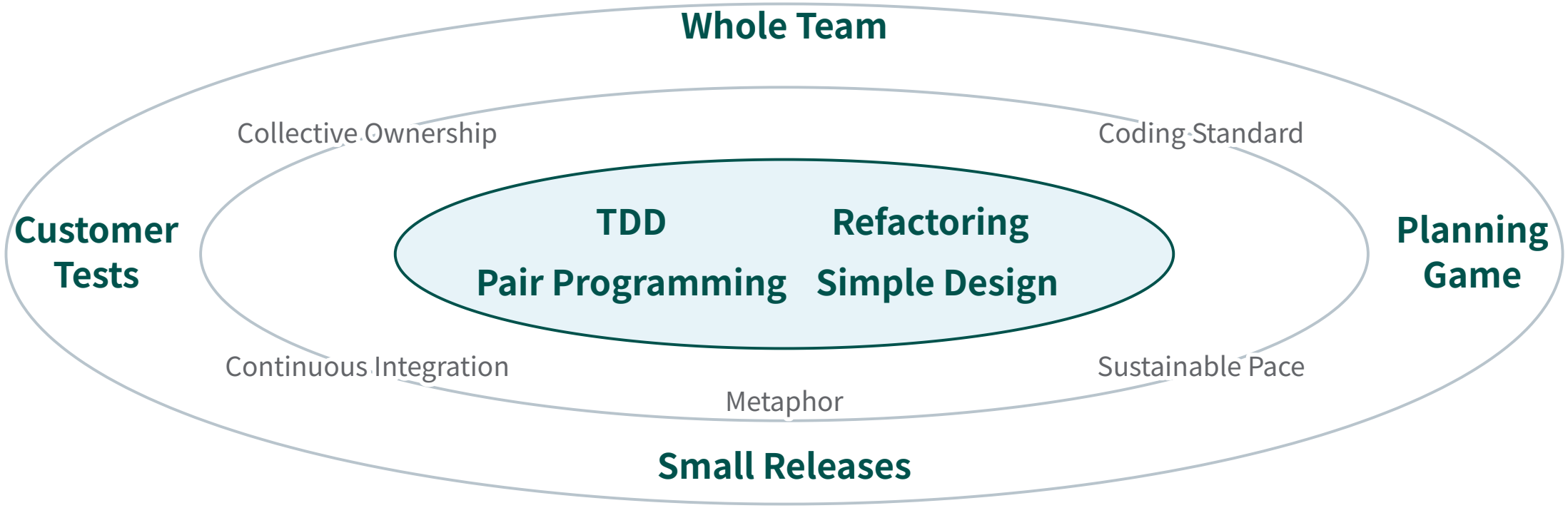
Respond to changing requirements with short feedback and practices that support each other.

Extreme Programming connects customer participation, testing, collaboration and design.

Paraphrase • Extreme Programming Explained • publisher

© 2026 Inspire | Confidential

Practices that support each other



TDD = Test-Driven Development.
The layers show scopes that support each other, not a fixed sequence.

© 2026 Inspire | Confidential

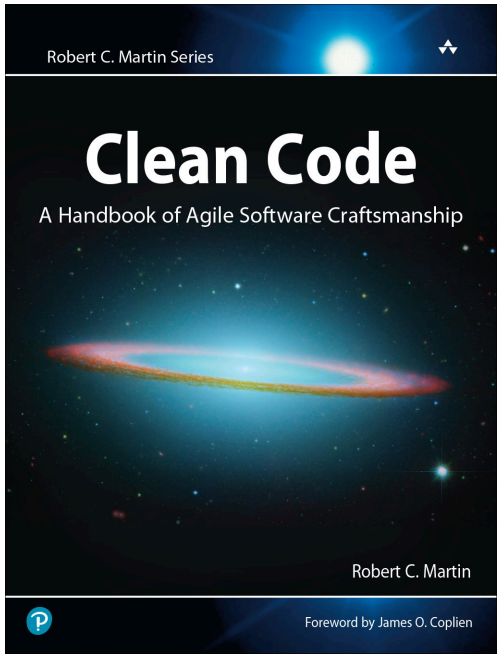
XP Core Practices



Tests protect refactoring. Refactoring keeps design clear.
Pair programming adds shared implementation and questioning.

© 2026 Inspire | Confidential

Robert C. Martin



First edition • 2008
Publisher cover: Prentice Hall / InformIT

Published 1 August 2008

Readable code helps people judge a design and keep changing it.

Clean Code trains readers to read, evaluate and improve code. Think of the Day 1 Context Map.

Paraphrase • Clean Code • publisher

© 2026 Inspire | Confidential

Kent Beck



Photo: Sbence92 • 8 April 2025 • CC0

Published 25 June 2025

He delegates implementation and still checks complexity, tests and design.

His B+ Tree experiment still had unexpected complexity. He intervened when the agent took shortcuts.

Paraphrase • Augmented Coding: Beyond the Vibes

© 2026 Inspire | Confidential

Simon Willison



Photo: Paul Downey • 2008 • CC BY 2.0
Cropped

Published 23 February 2026

Have the agent see a meaningful test failure before implementing the behavior.

Ask what the failing test detected, as well as whether it now passes.

Paraphrase • Red/green TDD • author's guide

© 2026 Inspire | Confidential

Birgitta Böckeler



Author photo: MartinFowler.com

Published 10 August 2026

A small exploration found no clear quality gain from TDD inside the agent loop.

Limited scope; this does not show that TDD is useless. She focuses on results, regression checks and human judgment points.

Paraphrase • TDD inside the agent loop • MartinFowler.com

© 2026 Inspire | Confidential

Robert C. Martin

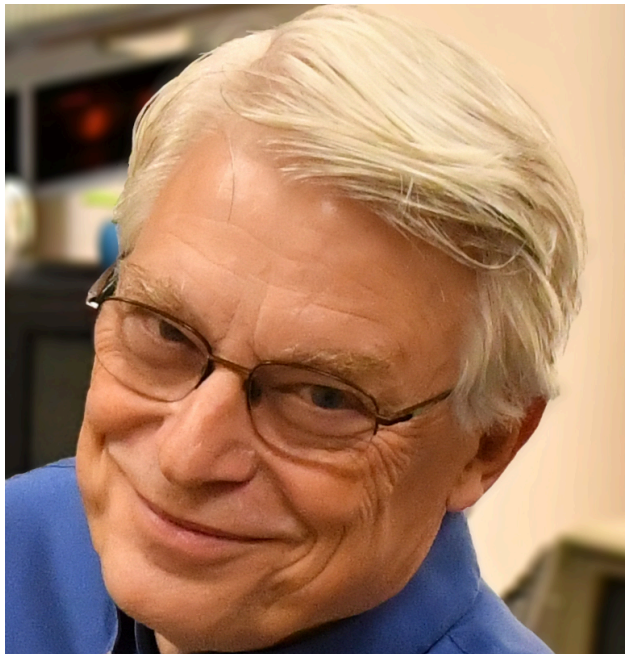


Photo: Angelacleancoder • 22 January 2020
CC BY-SA 4.0 • Cropped

Published 24 September 2026

He reports rarely reading generated code, using tests, behavior checks and architecture supervision.

He still emphasizes human responsibility and Clean Code principles. The strategy continues to evolve.

Paraphrase • Aviator interview

© 2026 Inspire | Confidential

Different workflows, questions to test

What is changing?

More implementation can be delegated.

People explore how much code to read and how agents should use TDD.

What do we need to check?

Who confirms the rule?

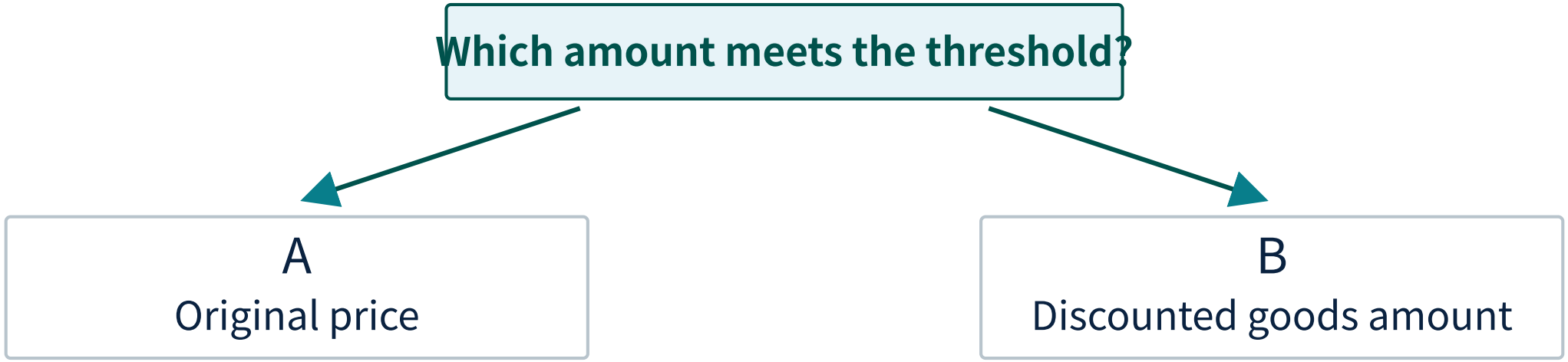
Who explains the change?

What did the feedback actually check?

Take these questions back to the Day 1 examples.

© 2026 Inspire | Confidential

Which amount decides free shipping?



One order: original price meets it; discounted amount does not.

A and B produce different decisions. Who can confirm the rule?

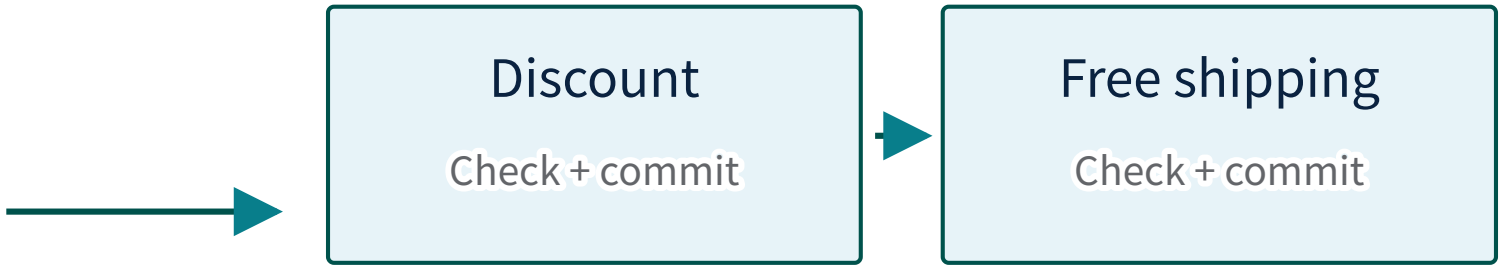
Candidate interpretations only. Use the confirmed Day 1 answer to choose the expected result.

© 2026 Inspire | Confidential

Read the difference. Choose the scope.

A possible mixed diff

```
+ discount behavior
+ free-shipping behavior
~ structure changes
? unrelated feature
```



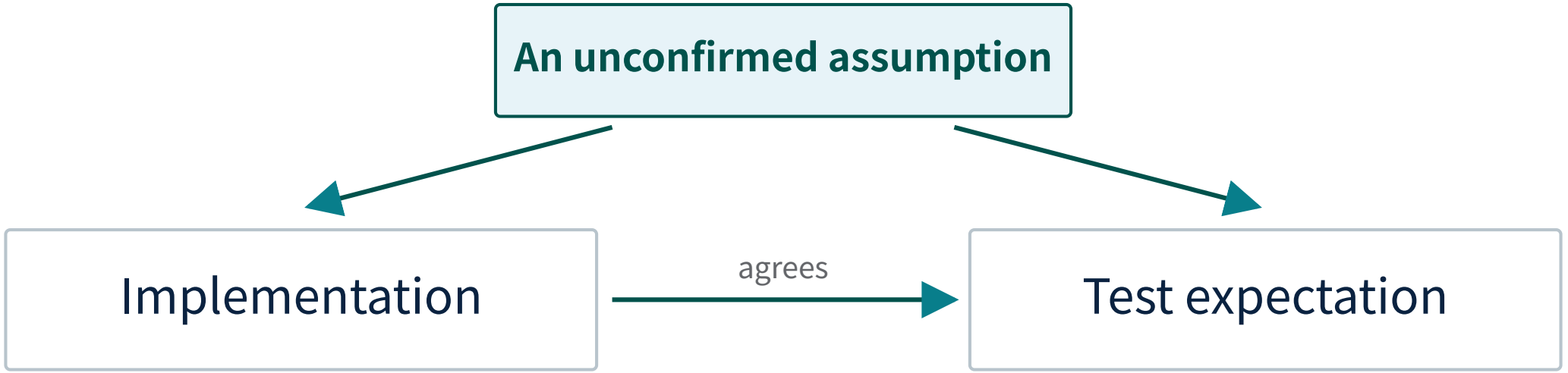
Check dependencies before splitting.

A small commit is not yet a small release.

Keep the accepted change small enough to understand and check.

© 2026 Inspire | Confidential

All tests pass. What do they prove?



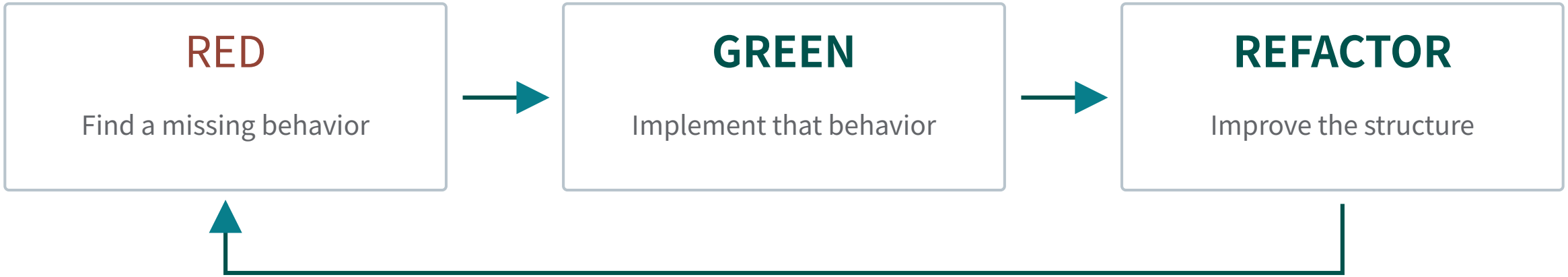
A green result can repeat the same mistake.

Trace the expected result to a confirmed rule and an example.

© 2026 Inspire | Confidential

Use a meaningful failure to guide a change

Confirmed behavior gives the test a trustworthy expectation.

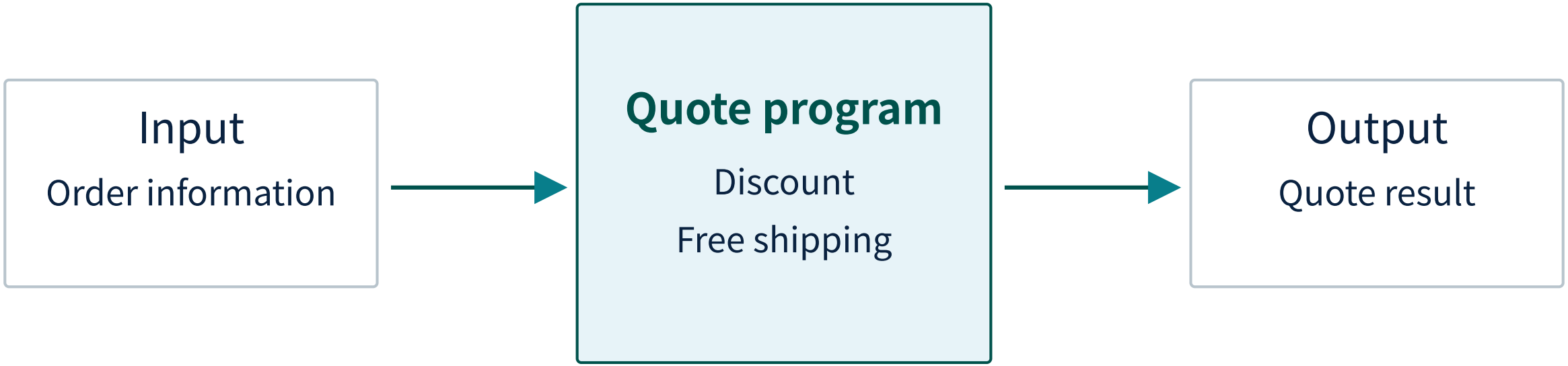


Explain the failure. Explain the change. Check again.

We learn TDD to connect behavior, feedback and design.
Asking an agent to “use TDD” is not proof of quality.

© 2026 Inspire | Confidential

Can you still explain your Context Map?



Can you explain responsibilities and the behavior that should stay the same?

Refactoring improves structure while preserving behavior.

© 2026 Inspire | Confidential

What new evidence does review bring?

Two people or two agents

Same assumption?

Agreement alone is not new evidence.



What does review add?

A missed rule

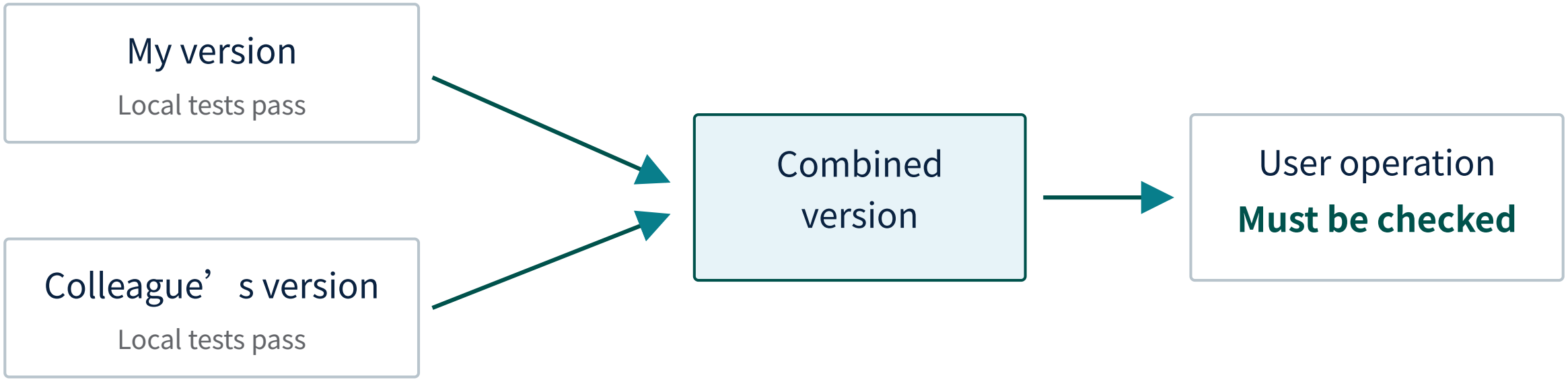
A counterexample

An actual run

AI assistance is not automatically equivalent to pair programming.

© 2026 Inspire | Confidential

Check the version people will use

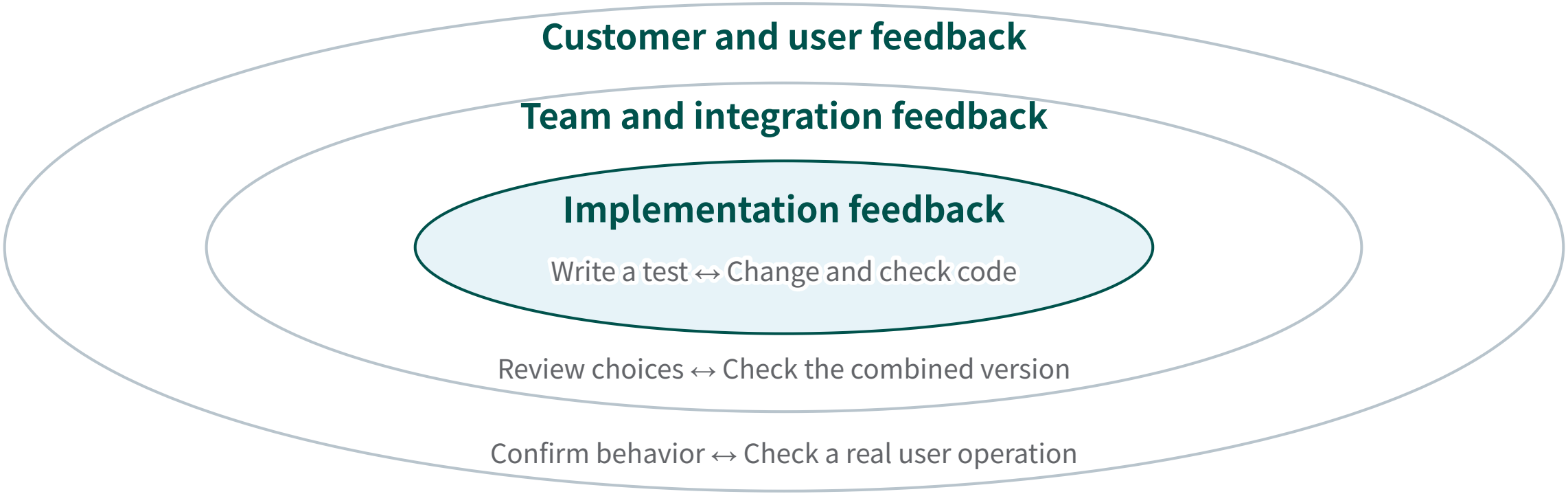


Local evidence and integrated evidence answer different questions.

Continuous Integration (CI): combine changes often, then build and check them.
Delivery also needs a selected version, impact assessment and recovery plan.

© 2026 Inspire | Confidential

Fast local feedback needs wider checks



© 2026 Inspire | Confidential

What evidence is missing?

1. What do you need to know?
2. Which practices help you find out?
3. Who decides whether to continue?

“I would first … because we still need …”

© 2026 Inspire | Confidential

The code and tests agree

AI wrote the free-shipping code.
AI wrote the tests. They all pass.

But nobody has confirmed which amount decides free shipping.

Would you accept it? What evidence do you need first?

© 2026 Inspire | Confidential

The change passes checks

Discount and free shipping work.
AI also changed much of the structure.

You cannot explain each part of the change.
You can only forward the AI summary.

What would you do before accepting the change?

© 2026 Inspire | Confidential

Local tests pass. The user operation fails.

Your tests pass.
Your colleague’ s tests pass.

After combining the changes, the user cannot complete the operation.

What would you report? What would you check next?

© 2026 Inspire | Confidential

Engineering skills help us guide and check AI work

01 Agree on what the software should do

Ask questions. Confirm examples.

02 Use small changes and repeatable checks

Compare changes. Run tests. Check integration.

03 Give AI tasks you can check

State the goal. Limit the scope. Check the result.

Engineering discipline gives us more room to explore useful solutions with AI.

© 2026 Inspire | Confidential

Why we practice these skills now

Today	Git • Code Review • Unit tests
Next	Test analysis • TDD • Refactoring
Later	Integration • Delivery • Teamwork

Today’ s practice is without AI.
Make the decisions yourself so you can later direct and check AI work.

© 2026 Inspire | Confidential

Why read a diff and make small commits?

Which judgment do they help you make?
What do they still not prove?

Think for 25 seconds.
Give one specific benefit and one limit.

© 2026 Inspire | Confidential

Compare two states

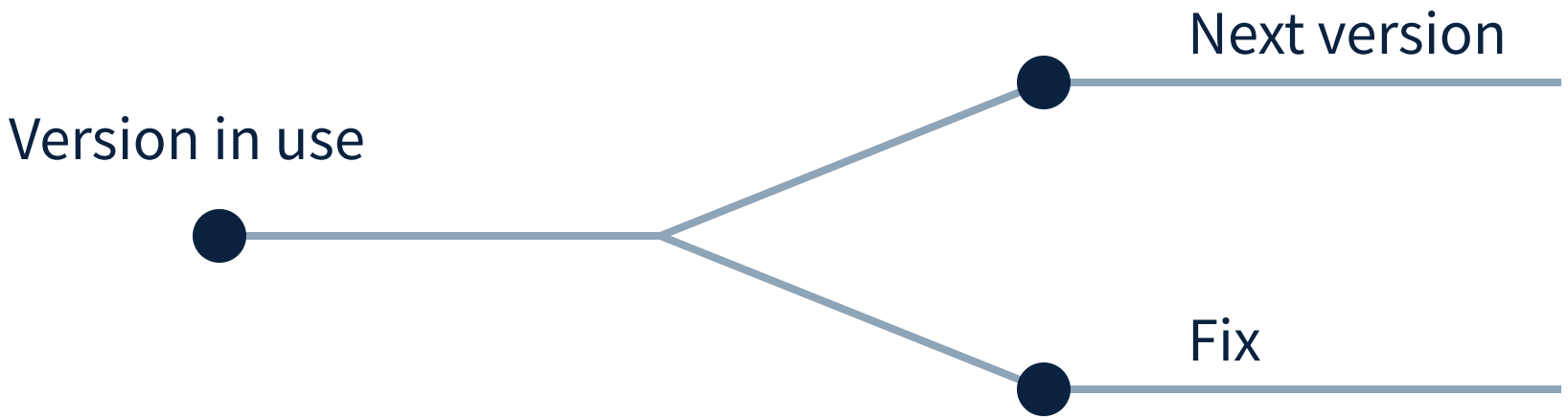


Observe the change before deciding how to save it.

© 2026 Inspire | Confidential

Software Keeps Changing

A team can maintain the version in use while developing the next version and fixing a problem.



We need to identify project states and the relationships between them.

© 2026 Inspire | Confidential

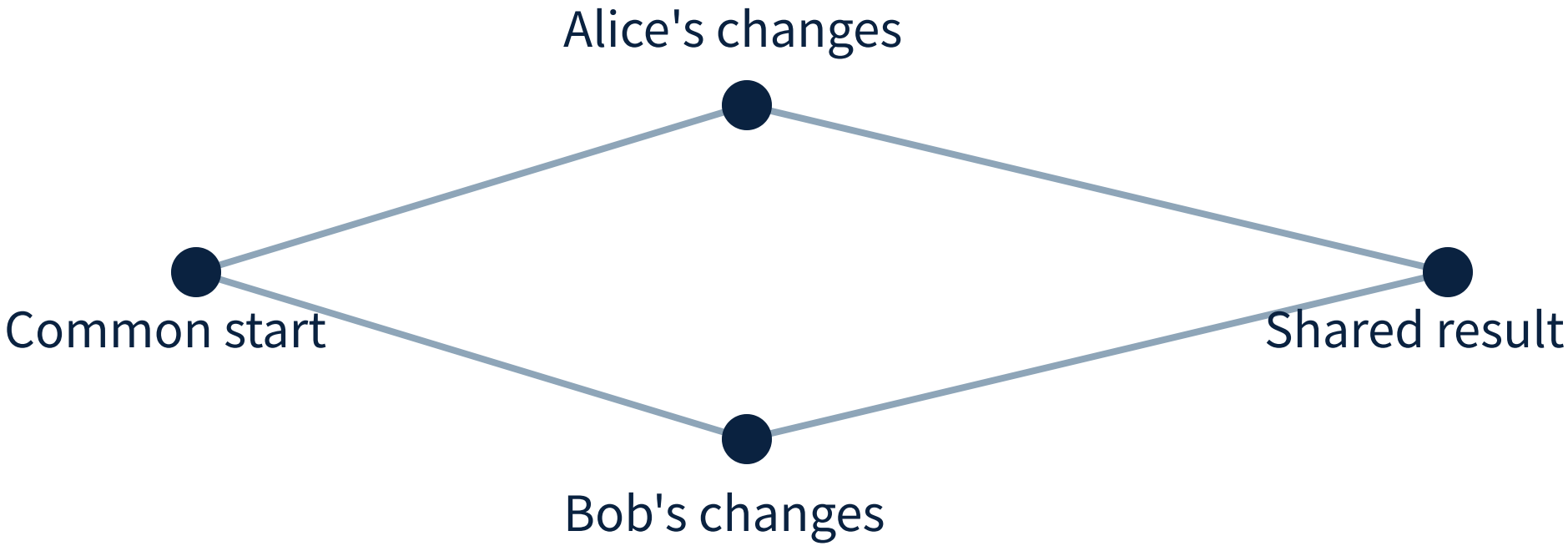
Recorded Versions Give Us a Reference

Identify	Which version did the user run? Which version did we test?
Compare	What changed between two recorded states?
Trace and recover	Where did the change begin? Can we retrieve earlier content?

Recording a version preserves a reference. Checks establish what works in that version.

© 2026 Inspire | Confidential

Independent Work Can Join a Shared Version



Version control supports comparison and integration. The combined behavior still needs checks.

© 2026 Inspire | Confidential

A Repository Records Versions and History

A local repository lives on your computer.

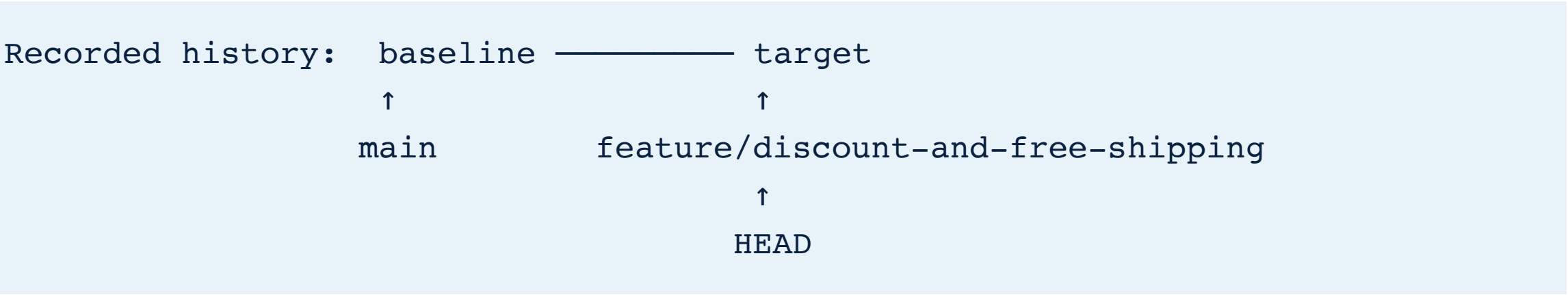
Commit	A snapshot of tracked project files, with a unique ID.
History	Parent commits connect the recorded states.
Commit information	Author and message help explain the record.

A diff shows changes between states. A commit represents the project state.

© 2026 Inspire | Confidential

Branches Name a Position in History

A branch name can move. A commit ID still identifies the same record.



HEAD selects the current branch here. A new commit advances that branch. Other branch names keep their positions.

© 2026 Inspire | Confidential

Working Files Hold Your Current Edits

The working tree is the project you open and edit.

Action	What changes?
Save a file in the editor	The working file
Read an earlier commit	You can compare against a recorded project state

Saving an edit leaves existing commits and branch positions unchanged.

© 2026 Inspire | Confidential

The Staging Area Prepares the Next Snapshot

Working tree	All current file content, including ongoing edits.
Staging area	The project content prepared for the next commit. Also called the index.
Commit	Records the prepared snapshot and advances the current branch.

git add copies the selected content at that moment into the index. Unchanged tracked files remain part of the snapshot.

© 2026 Inspire | Confidential

Which Content Will the Commit Record?

The working files contain both practice changes. The staging area contains only the first selected change.

What will the new snapshot contain?
Where will the remaining change stay?

© 2026 Inspire | Confidential

Selection Changes the Prepared Snapshot

Stage of the practice	HEAD	Index	Working files
After mixed reset	Baseline	Baseline	Full target
After partial staging	Baseline	Baseline + selection	Full target
After commit	New commit	New commit	Full target

The feature branch advances. main stays at the baseline. A local commit stays in your local repository until published.

© 2026 Inspire | Confidential

Mixed Reset Keeps the Working Files

In this prepared clone, the target has one parent, and that parent is main.

```
git rev-parse HEAD           # record the full target SHA
git reset --mixed HEAD~1     # run once at the verified start
```

Current feature branch	Index	Working files
Moves to the baseline	Returns to the baseline	Keeps the target content

The recorded target SHA still identifies the original commit. HEAD~1 means the current commit's parent.

© 2026 Inspire | Confidential

Reset Options Affect Different States

Option	Current branch	Index	Working files
`--soft`	Moves	Keeps content	Keeps content
`--mixed`	Moves	Resets to target	Keeps content
`--hard`	Moves	Resets to target	Replaces tracked content

Today: mixed reset once in the new practice clone, before personal commits.

Keep Day 1 work separate. Do not run hard reset or push the reset history.

© 2026 Inspire | Confidential

Every Diff Compares Two States

Command	Baseline	Compared state
`git diff`	Index	Working tree
`git diff --cached`	HEAD commit	Index
`git diff main <target-sha> --`	Recorded main	Recorded target

After partial staging, plain diff shows the changes still outside the index. Untracked file content needs a separate check.

© 2026 Inspire | Confidential

What Will the Next Commit Add?

Some changes have entered the index. Other changes remain in the working tree.

Which diff checks the next commit?
Name the two states it compares.

Next: connect the same model to the actual terminal output.

© 2026 Inspire | Confidential

Demo: Check the Starting Point

Use the new class practice clone with a clean starting state.

```
git status --short
git switch feature/discount-and-free-shipping
git branch --show-current
git log --oneline --graph main feature/discount-and-free-shipping
git rev-parse HEAD
```

Verify one target commit above main, with main as its only parent. Record the full SHA before reset. Stop if a check differs.

© 2026 Inspire | Confidential

Demo: Predict, Reset and Check

```
git reset --mixed HEAD~1
git log --oneline -3
git status --short
git diff --cached
git diff
git diff <target-sha> --
```

Expected: HEAD at main, empty cached diff, both changes in plain diff, no difference from the recorded target's files.

© 2026 Inspire | Confidential

Demo: Explain the Promotion Change

main

```
        BigDecimal memberPrice = Money.memberPrice(original, member);
        BigDecimal promotion = Money.amount("0.00");
        BigDecimal goods = memberPrice.subtract(promotion);
```

feature/discount-and-free-shipping

```
        BigDecimal memberPrice = Money.memberPrice(original, member);
        BigDecimal promotion = memberPrice.compareTo(Money.amount("100.00")) >= 0
                ? Money.amount("20.00") : Money.amount("0.00");
        BigDecimal goods = memberPrice.subtract(promotion);
```

QuoteCalculator.java • Selected committed source comparison.

Trace memberPrice into promotion, then follow how goods uses the result. Leave the changes unstaged.

© 2026 Inspire | Confidential

Your Turn: Establish the Checked State

1. Check your directory, clean status, branch and single-commit relationship.
2. Record the full target SHA. Run the demonstrated mixed reset once.
3. Check HEAD, cached diff, plain diff and the recorded target comparison.

Use your own new class clone. Keep unexpected output and ask your coach before continuing. Begin reading when your checks pass.

© 2026 Inspire | Confidential

Your Turn: Explain the Shipping Change

Read your current diff in the IDE or terminal.

- Locate the old shipping calculation and its replacement.
- Follow the amount used by the new calculation.
- Explain how that value depends on the promotion calculation.

Point to the changed and surrounding code. Keep all changes unstaged and uncommitted.

© 2026 Inspire | Confidential

Does Your Explanation Match the Code?

Return to the question raised during practice.

Which line supports the explanation?
Where does its input come from?

Keep your diff and target SHA. Next: describe the purpose of each change in a commit message.

© 2026 Inspire | Confidential

Conventional Commits Describe the Purpose

```
type(scope): description
feat(profile): add a display name
feat: add a display name
```

type	The purpose of the change.
scope	An optional affected area.
description	A short, specific account of what changes.

© 2026 Inspire | Confidential

Context Explains the Type

`feat`	`fix`	`refactor`
Add a new capability.	Meet an existing agreement.	Change structure. Preserve observable behavior.
New agreement	Missed requirement	Same behavior

Read the agreement → inspect the change → check the evidence.

A commit message states an intent. The type alone does not prove what the change does.

© 2026 Inspire | Confidential

Same Edit. Which Purpose?

“Trim spaces before saving an email.”

A • New request

Saving the original input met the previous agreement. A new request now adds trimming.

B • Existing requirement

The agreement already required trimming. The implementation missed that step.

`feat` `fix` `refactor` Mixed purposes Insufficient information

Independent hypothetical examples. Choose a type for A and B. Point to the agreement that supports each choice.

© 2026 Inspire | Confidential

What Can You Conclude So Far?

C

Extract email cleanup

Move duplicated trim calls into a shared method. Keep call positions, validation and rules.

What stays the same?

D

Extract email cleanup

The author claims a structural change. The method body and full request are still missing.

What must you inspect?

E

Improve email handling

No request, diff or failure is available.

What is missing?

Independent hypothetical examples. Choose feat, fix, refactor, mixed purposes or insufficient information. Explain your reason.

© 2026 Inspire | Confidential

One Diff. Two Different Agreements.

Hypothetical code · the same edit in A and B

```
- repository.save(email);
+ repository.save(email.trim());
```

A · New requirement

feat

Add a capability that was not required before.

B · Existing requirement

fix

Repair the missed trimming requirement.

Compare your reasons, then advance to reveal A and B in turn. The verb “trim” alone does not determine the type.

© 2026 Inspire | Confidential

C: Follow the Behavior Through the Extraction

Hypothetical code • both callers use this method

```
String cleanEmail(String email) {
    return email.trim();
}
```

Callers now save `cleanEmail(email)`.

What stays the same?

- The original `email.trim()` expression
- Validation and call order
- The processing rule

`refactor`

Shared structure; preserved behavior.

Explain what preserves the behavior before revealing the supported purpose. Check against the previous version.

© 2026 Inspire | Confidential

D: Find the Added Behavior

Hypothetical code • empty strings were allowed before

```
String cleanEmail(String email) {
    String cleaned = email.trim();
    if (cleaned.isEmpty()) {
        throw new IllegalArgumentException(
            "email required");
    }
    return cleaned;
}
```

Two independent tasks

`refactor`

Extract the existing cleanup.

`feat`

Add rejection when the cleaned email is empty.

E remains open: request and diff still missing.

Which lines move existing logic? Which lines add a rule? Explain both purposes before advancing.

© 2026 Inspire | Confidential

Your Turn: Describe Your Two Changes

Return to your QuoteCalculator diff. In your existing notes, draft a message for promotion and another for shipping.

- What capability does each change add to the baseline?
- Does each description match the code you plan to select?

Use specific descriptions. Keep missing information visible. Next: select the matching content for a commit.

© 2026 Inspire | Confidential

Demo: Select the Promotion Change

First check the retained state and compare the working files with the recorded target.

```
git branch --show-current
git status --short
git diff --cached
git diff
git diff <target-sha> --
git add -p src/main/java/training/quote/QuoteCalculator.java
```

Keep the current feature branch, an empty index diff and both practice changes in the working tree.

© 2026 Inspire | Confidential

A Hunk Groups Nearby Text Changes

One hunk can contain more than one purpose.

add -p choice	Meaning
`s`	Try splitting the hunk into smaller pieces
`y`	Stage the displayed piece
`n`	Leave the displayed piece outside the index

Read the actual code before each choice. Check cached diff for the selected promotion and plain diff for the remaining shipping change.

© 2026 Inspire | Confidential

Demo: Correct a Selection Before Commit

The staged diff now includes both purposes. Cancel the file's staging and select again.

```
git restore --staged src/main/java/training/quote/QuoteCalculator.java
git diff --cached
git diff
git diff <target-sha> --
```

This resets the whole file's index content to HEAD. Working files stay unchanged. Reselect promotion after checking the output.

© 2026 Inspire | Confidential

Demo: Check the Selection and Commit

```
git add -p src/main/java/training/quote/QuoteCalculator.java
git diff --cached
git diff --cached --check
git commit -m "feat: apply the agreed promotion"
```

The message describes the selected change.

--check detects some whitespace problems. Business behavior needs separate evidence.

© 2026 Inspire | Confidential

Demo: Inspect the Commit and What Remains

```
git log --oneline -3
git show HEAD -- src/main/java/training/quote/QuoteCalculator.java
git diff --cached
git diff
git diff <target-sha> --
```

The saved commit contains promotion. Plain diff retains shipping. Cached diff and the original-target comparison are empty. Keep your directory, SHA and draft messages through the break.

© 2026 Inspire | Confidential

Your Turn: Commit the Promotion Change

1. Return to your practice directory. Confirm the branch, target SHA and retained changes.
2. Select promotion with add -p. Read both diffs and check your message.
3. Run the whitespace check, commit, then inspect the saved change.

Explain the selected purpose and the shipping change left in plain diff. Keep unexpected states and ask for help.

© 2026 Inspire | Confidential

Your Turn: Commit the Remaining Shipping Change

Work independently with your own message.

1. Read what remains in plain diff.
2. Select it, inspect the staged diff and check the message.
3. Commit, then inspect what you saved and anything left in the project.

If the remaining diff differs from the demonstration, inspect the first commit. Check the source and content of unexpected files before adding them.

© 2026 Inspire | Confidential

Your Turn: Inspect Every Commit

```
git log --oneline main..HEAD
git show <commit-sha> -- src/main/java/training/quote/QuoteCalculator.java
git status --short
git diff --cached
git diff
git diff <target-sha> --
```

Copy each new commit SHA from your log and inspect it. Use the original target SHA for the last comparison. Different commit IDs are expected.

© 2026 Inspire | Confidential

Correct an Uncommitted Selection or Keep the Issue

Known staging error	Read it, use restore --staged, check the retained files and select again.
Already committed?	Keep the commit and question. Ask the coach to check how this copy can continue.
Unexplained state	Stop and keep the original directory. A coach checks whether recovery is needed.

Check file preservation and each message separately. A clean working tree alone does not establish correct behavior.

© 2026 Inspire | Confidential

Keep Your Result and Any Unfinished Work

- Retain your commits, diffs and original target SHA.
- Record unresolved questions, current directory and branch, remaining changes and help received.
- If a recovery copy was needed, record its reason and checked starting point.

Next: collaboration in a separate repository.

© 2026 Inspire | Confidential

Local Commits Become Shared Work

You have saved your work. How can another person continue from it?



Alice and Bob are hypothetical demonstration IDs. Each repository has its own branches.

© 2026 Inspire | Confidential

Three Branch Positions from Bob's View

Name	Location	What it tells Bob
main	Bob's repository	My current local branch
origin/main	Bob's repository	My last known position of shared main
main on origin	Shared repository	The team's actual published branch

origin is a configured remote name.
origin/main is a local record that can be out of date.

© 2026 Inspire | Confidential

Alice's New Commit Exists Locally

Alice's computer



main

origin/main

Shared repository



main

Bob's computer



main

origin/main

Alice has committed. Who can read A now?

© 2026 Inspire | Confidential

Push Requests Publication

```
git push origin main
```

After Alice's successful push	main	origin/main
Alice's repository	A	A
Shared repository	A	—
Bob's repository	P	P

With the normal origin mapping, Alice's origin/main updates too. Bob has had no communication yet.

© 2026 Inspire | Confidential

Fetch Updates Bob's Local Record

```
git fetch origin
```

Shared repository

main = A



Bob's repository

origin/main = A

main stays where it was

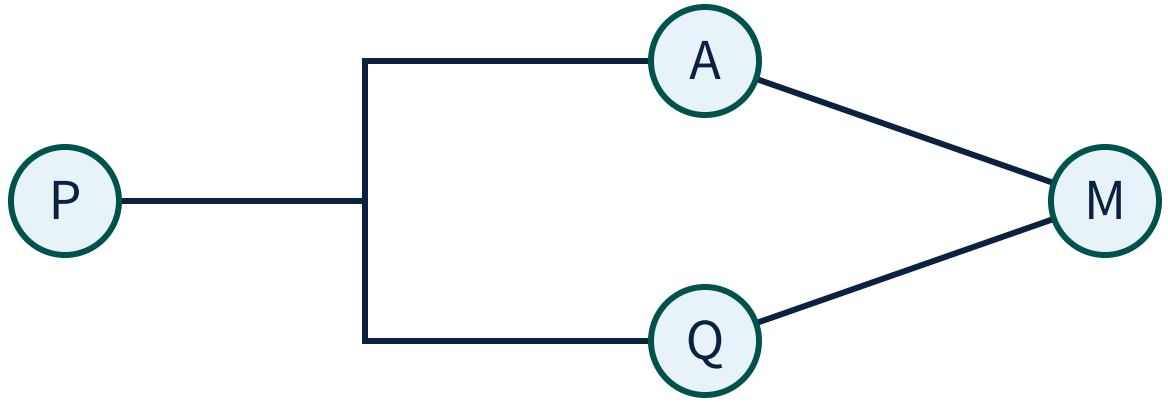
Working files stay unchanged

Bob now has the data to inspect and compare A. Integration is a separate action.

© 2026 Inspire | Confidential

Merge Combines the Histories Locally

```
git merge --no-ff --no-edit origin/main
```



Bob's main = M

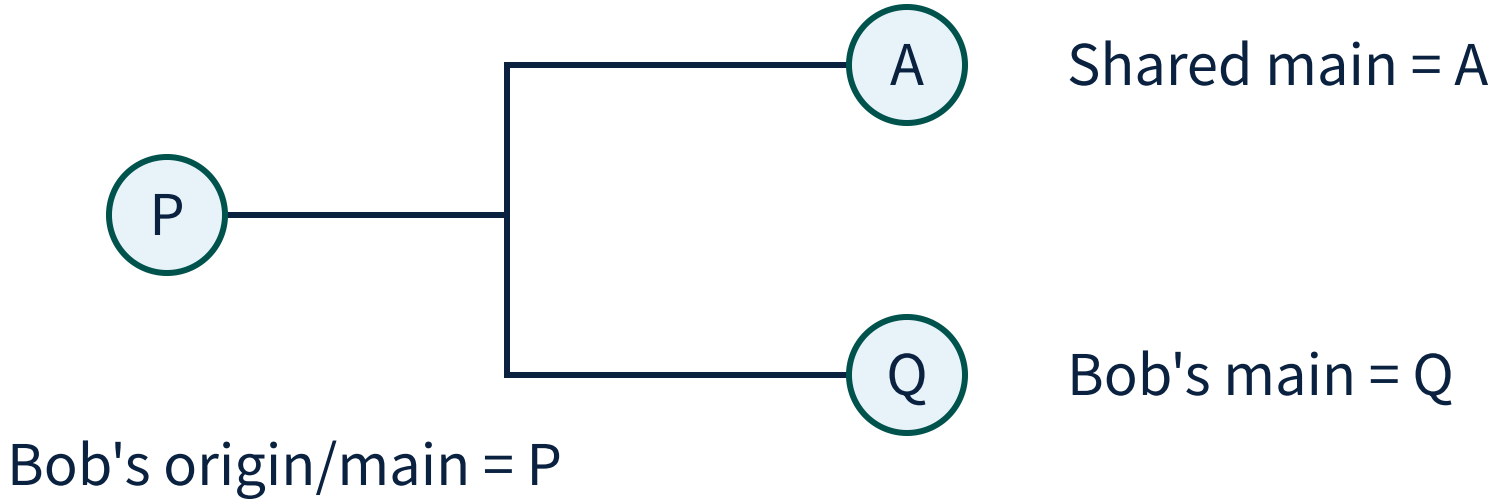
Bob's origin/main = A

Shared main = A

A successful merge updates Bob's current branch, index and working files. M still needs publication.

© 2026 Inspire | Confidential

Different Files Can Still Cause a Push Rejection



A is outside Q's history. Bob must integrate the published progress before publishing Q.

The rejection checks commit relationships. It does not establish a content conflict.

© 2026 Inspire | Confidential

What Moves after Bob Fetches?

Before fetch:

Bob's main = Q, Bob's origin/main = P, shared main = A.

After a successful fetch, where are the three references?

Think individually for 20 seconds. Be ready to explain one position.

© 2026 Inspire | Confidential

Fetch, Merge and Push Are Separate Actions

Bob's stage	Local main	origin/main	Shared main
Push rejected	Q	P	A
Fetch succeeds	Q	A	A
Merge creates M	M	A	A
Push succeeds	M	M	M

M preserves both histories.
Alice still needs to obtain the new shared version.

Another publication before Bob's push can cause a new rejection.

© 2026 Inspire | Confidential

A Rejected Push and a Content Conflict Differ

During push	The shared branch has advanced. Obtain and integrate its progress.
During merge	Git may combine the text automatically, or ask you to decide the content.
Merge stops	M does not exist yet. Inspect the files and index before completing it.

Next: two demonstration copies. Track whose branch changes at each action.

© 2026 Inspire | Confidential

Two Demonstration Copies Start Together

```
git clone <demo-repository-path> fleet-demo-alice
cd fleet-demo-alice
git branch --show-current
git remote -v
git status --short
git rev-parse --short HEAD
```

Prepared disposable demo repository. Check main, origin, a clean state and the same starting commit as Bob.

© 2026 Inspire | Confidential

Alice Publishes Her Introduction

Create and read `introductions/alice.txt`.

```
git add introductions/alice.txt
git diff --cached
git commit -m "docs: add Alice introduction"
git push origin main
```

A file in the editor, a local commit and a published change are different observations. Check the shared result.

© 2026 Inspire | Confidential

Bob's Different File Can Merge Automatically

Bob commits `introductions/bob.txt` from the old baseline, then tries to push.

```
git push origin main
# After confirming that shared main has advanced:
git fetch origin
git merge --no-ff --no-edit origin/main
```

Read the actual output. If merge succeeds, inspect both introduction files, then push. No extra conflict-resolution commit is needed.

© 2026 Inspire | Confidential

Bob Has Published. Alice Can Still Be Behind.

```
git log --oneline --graph -4
git rev-parse --short main
git rev-parse --short origin/main
# In Bob's copy, read the shared branch:
git ls-remote origin refs/heads/main
```

Which copy contains both introductions now?

Compare actual terminal results. Alice has not fetched since her own publication.

© 2026 Inspire | Confidential

Alice Actively Obtains the Shared Result

Alice has a clean copy with no unfinished merge or unpublished local work.

```
git status --short
git pull --ff-only origin main
git rev-parse --short HEAD
git status --short
```

This fetches and allows only a fast-forward update. If histories diverge, it stops for inspection.

© 2026 Inspire | Confidential

The Saved Result Will Reconnect Us after Lunch

Both copies now contain the two introductions at the same checked commit.

> Save the published merge SHA and terminal output.

After lunch: the selected class Lab. Keep the morning quote project in its own directory.

© 2026 Inspire | Confidential

Your Class Lab and Your Own ID

Class Lab: `<selected-class-lab-url>`

Your unique ID: `<your-roster-id>`

```
git clone <selected-class-lab-url> fleet-lab
cd fleet-lab
git branch --show-current
git remote -v
git status --short
```

The instructor supplies the selected class's approved values. If fleet-lab already exists, ask your coach to inspect it first.

© 2026 Inspire | Confidential

Your Published Introduction Is Your Ticket

Create `introductions/<id>.txt` with one sentence. Read it first.

```
git add introductions/<id>.txt
git diff --cached
git commit -m "docs: add my introduction"
git push origin main
```

Use your own ID everywhere. An untracked file may not appear in ordinary git diff. Read the staged content before committing.

© 2026 Inspire | Confidential

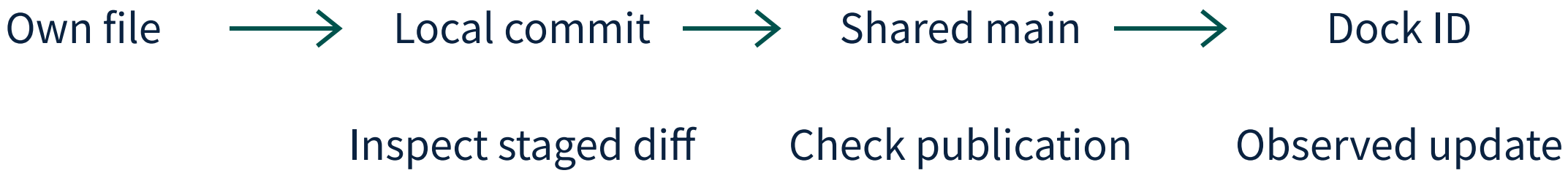
The Error Determines the Next Action

Authentication	Keep the error. Ask the coach to check access and identity.
Network	Keep the error. Check the connection with the coach.
main advanced	Fetch, merge, inspect the result and retry the push.

A same-name introduction conflict needs an ID and source check before editing.

© 2026 Inspire | Confidential

The Dock Shows Published Introductions



Find your introduction on shared main.
Click your Dock ID to read it. Close with ×, Esc or an outside click.

No unfinished merge, uncommitted files or unpublished local commits. If the display lags, check its update time and the shared commit.

© 2026 Inspire | Confidential

Two Prepared Commits Change the Same Group Line

Disposable demo copy	boarding.txt	State at the start
fleet-conflict-alice	`group-1: [alice]`	Alice has published
fleet-conflict-bob	`group-1: [bob]`	Bob has committed locally

Both commits began from the same clean, empty-list baseline after the earlier demo.

Hypothetical IDs and group. These prepared copies use the isolated demo repository.

© 2026 Inspire | Confidential

Push Rejects. Merge Then Finds a Content Conflict.

Current terminal: `fleet-conflict-bob`.

```
git push origin main
git fetch origin
git merge --no-ff --no-edit origin/main
git status
```

Confirm the push error first. When merge stops, use status to find boarding.txt. Read both versions before choosing the result.

© 2026 Inspire | Confidential

The List Rule Determines the Result

```
<<<<<<< HEAD
group-1: [bob]
=======
group-1: [alice]
>>>>>>> origin/main
```



Published order first

```
group-1: [alice, bob]
```

Then the local ID once

Hypothetical conflict. Keep one valid group line and its fixed delimiters. The correct resolution depends on the file's rules.

© 2026 Inspire | Confidential

A Resolved File Still Needs a Merge Commit

Remove the markers. Read the entire list and its neighboring groups.

```
git add boarding.txt
git diff --cached
git diff --cached --check
git status
```

Check each ID once, published order and the fixed separators. The merge remains in progress until commit.

© 2026 Inspire | Confidential

The Checked Merge Can Now Become Shared

```
git commit -m "docs: preserve all boarding IDs"
git push origin main
git status
```

If you kept only one side, whose work would disappear?

If main advances again, fetch and merge again. Keep the original ID commit. Now return to your class Lab and your real IDs.

© 2026 Inspire | Confidential

One Command Path for Both Rounds

Inspect your intended file, stage it, inspect again and commit.

```
git diff
git add boarding.txt
git diff --cached
git commit -m "docs: add my boarding ID"
```

Stop after commit. Wait for the instructor's push window.

Round 2 uses reboarding.txt and a reboarding message. Keep both round files.

© 2026 Inspire | Confidential

Round 1 Starts from One Shared Baseline

Start boarding opens Boarding. Prepare your first boarding commit. Pushes paused. Announced baseline: `<round-1-sha>`.

```
git branch --show-current
git status --short
git pull --ff-only origin main
git rev-parse --short HEAD
```

First 4 minutes of the 15-minute step: confirm main, clean state and matching HEAD. Then add only your ID to your group's list and commit before pushes open.

© 2026 Inspire | Confidential

Pushes Open after Everyone Has a Local Commit

```
git push origin main
# If rejected because main has advanced:
git fetch origin
git merge --no-ff --no-edit origin/main
```

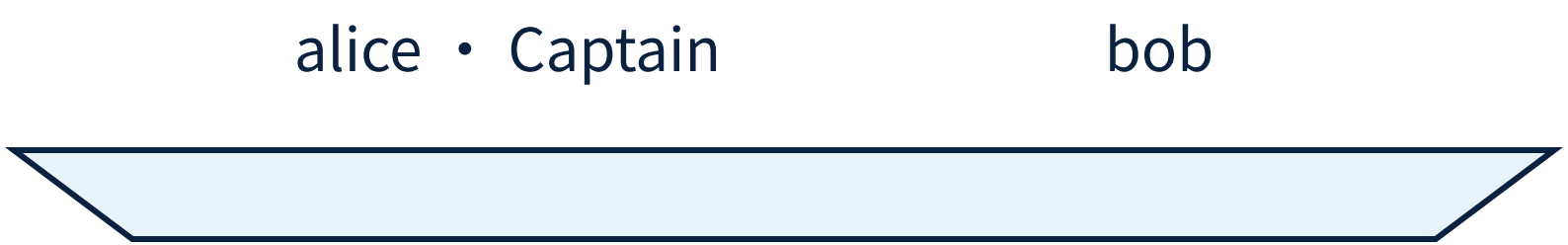
Read who has joined. Preserve their order. Add your ID once.

Next 8 minutes: inspect automatic merges or resolve content conflicts, check, complete and retry. You operate your own keyboard.

© 2026 Inspire | Confidential

The First Valid Published ID Is the Captain

```
# Hypothetical list, not the class roster
group-1: [alice, bob]
```



Last 3 minutes: confirm real group members once each and preserve order. A refresh batch, author name or timestamp does not choose the captain.

© 2026 Inspire | Confidential

At the Island, Collect Supplies and Board Again

Set sail for the island opens Reboarding at the supply island.
Collect supplies, then board the same boat again.

boarding.txt	Keep the completed list and original captain.
reboarding.txt	Each person prepares a reboarding commit from the new common baseline.
Captain last	Other members publish first. The captain keeps their prepared commit and waits.

This step has 20 minutes. Coaches prioritize anyone without an observed content-conflict attempt.

© 2026 Inspire | Confidential

Everyone Prepares before the Reboarding Window Opens

Pushes paused. Announced baseline: `<round-2-sha>`. Edit only your group's reboarding list after the update.

```
git pull --ff-only origin main
git rev-parse --short HEAD
git diff
git add reboarding.txt
git diff --cached
git commit -m "docs: add my reboarding ID"
```

First 4 minutes: use the same branch and clean-state checks as round 1. The captain also commits now, before anyone publishes.

© 2026 Inspire | Confidential

Non-captains Publish while the Captain Waits

Non-captains fetch, integrate and publish their own reboarding change.

Captain: keep your prepared local commit.
Wait for every other member in your group's valid shared reboarding list.

Next 10 minutes: captains may inspect or fetch, but wait before merge. Coaches observe the learner's actual conflict actions and help.

© 2026 Inspire | Confidential

The Captain Integrates and Boards Again Last

When all other members of your boat have returned:

```
git fetch origin
git merge --no-ff --no-edit origin/main
```

Keep the published order. Append your own ID once.
Check the staged result, complete the merge and push.

Next 4 minutes: each boat proceeds when ready. The coach observes the captain handling the conflict.

© 2026 Inspire | Confidential

Sail Again after the Crew Has Returned

All first-round crew back, captain last: Sail again becomes available.
It opens Voyage complete. The journey on the display is complete.

Check	What to confirm
Boarding list	Original members and captain remain
Reboarding list	Each real member once, captain last
Individual work	Coach observed your conflict action, result and help

Last 2 minutes: keep missing evidence visible. A complete display or an automatic merge does not establish a personal content-conflict attempt.

© 2026 Inspire | Confidential

Synchronize Every Computer

Voyage complete: now synchronize every computer.
Writes stop. Final shared commit: `<final-main-sha>`.

```
git pull --ff-only origin main
git rev-parse --short HEAD
git status
```

Different SHA or unfinished work? Keep the output and ask your coach to inspect it.

First 3 minutes of the 5-minute step: match the final commit, read both complete lists and check a clean working tree.

© 2026 Inspire | Confidential

The Fleet Display Shows an Observed Shared State

Display entry	`<selected-class-display-entry>`
Read together	Active stage, observed commit, last valid update and errors
Offline fallback	Use the static diagram and actual terminal checks. Keep remote evidence gaps visible.

Class-specific entry remains to be supplied. This slide does not poll a repository or establish that the live display is ready.

© 2026 Inspire | Confidential

The Fleet Completes When Everyone Can Continue

Shared lists	Personal attempts	Every computer
Real members once	Observed conflict work	Same final commit
Captain last to board again	Help recorded	Clean working tree

After the break: a change can merge. Should we accept it?

Voyage complete confirms the shared list journey only. Check all three kinds of evidence. Keep the Lab records for the transition to Code Review.

© 2026 Inspire | Confidential

The Exercise Has One Shared Recovery Allowance

Five minutes in total, used wherever a delay occurs.

> If entry uses 2 minutes, 3 minutes remain.

For repeated push rejections, the instructor can pause other pushes and let one learner finish their own integration.

Authentication and network errors need their own diagnosis. Pausing publication does not replace personal merge work.

© 2026 Inspire | Confidential

A Recovery Starts by Preserving the Attempt

Before abandoning an unfinished merge, save:

- `git status` and the conflicting file content
- The relevant commit IDs and the help received

> The coach checks that the merge began cleanly and your own ID was already committed.

Then the coach can explain `git merge --abort`. You perform it and verify the original ID commit, index and working files before retrying.

© 2026 Inspire | Confidential

Unfinished Work Stays Visible

If the state remains unclear or recovery exceeds the allowance, stop and retain the actual work.

Record the unfinished lists, conflict observations or final synchronization.

Keep existing contributions. Avoid force push, reset, deleting someone else's content or silently replacing the learner's copy to claim completion.

© 2026 Inspire | Confidential

A Merge Leaves One Question

We can combine the versions. Should we accept the change?

Which agreement and which result would support your decision?

© 2026 Inspire | Confidential

The Request Sets the Review Scope

Allowed change	Correct the amount used to decide free shipping.
Agreed behavior	Use goods payable after the member discount and the single promotion to decide free shipping. Otherwise keep shipping at 10.00. Keep the other quote behavior unchanged.
Comparison	checkpoint/free-shipping → feature/member-delivery-fee

Read the agreed behavior before judging the implementation.

© 2026 Inspire | Confidential

Which Changes Affect the Agreement?

QuoteCalculator.java

Removed

```
        BigDecimal shippingBasis = original;
        BigDecimal deliveryFee =
Money.amount("10.00");
```

Added

```
        BigDecimal shippingBasis = memberPrice;
        BigDecimal deliveryFee =
Money.amount("8.00");
```

QuotePrinter.java

Removed

```
        String text = "Order
quote\n"
        return text;
```

Added

```
        String quoteText = "Order
quote\n"
        return quoteText;
```

Compare the behavior and scope first. Then discuss clarity and maintenance.

© 2026 Inspire | Confidential

One Example Supports a Specific Concern

Confirmed input	Expected shipping / total	Saved candidate result shipping / total
50.00 • nonmember REVIEW-SHIPPING-E01	10.00 / 60.00	8.00 / 58.00

Saved run 2026-10-19 • Candidate supplied review version
Recorded classroom review example
Saved macOS run with JDK 27 / release 21; current version and classroom results still need checking.

This is a saved candidate result. Identify its version and source before citing it.

© 2026 Inspire | Confidential

A Comment Gives the Author a Next Action

QuoteCalculator.calculate • deliveryFee

The request keeps non-free shipping at 10.00. For REVIEW-SHIPPING-E01 (original 50.00, nonmember), expected shipping is 10.00 and total is 60.00.

The saved candidate returned shipping 8.00 and total 58.00. Keep the agreed fee, rerun this example on the updated version and attach its actual result.

Saved candidate supplied review version
Recorded classroom review example • 2026-10-19

Location • agreed behavior • supporting result • requested action

© 2026 Inspire | Confidential

Your Turn: Make the Comment Useful

> “The shipping logic is wrong. Please rewrite it.”

Use goods payable after the member discount and the single promotion to decide free shipping. Otherwise keep shipping at 10.00. Keep the other quote behavior unchanged.

REVIEW-SHIPPING-E02: original 110.00, nonmember. Expected goods 90.00, shipping 10.00, total 100.00.

Candidate result: withheld for your judgment. Identify what you would check.

Write individually for 2 minutes. Use the request, the diff and a supplied example. Keep unverified claims as questions.

© 2026 Inspire | Confidential

What Does Each Result Support?

Confirmed input	Expected shipping / total	Saved candidate result shipping / total
110.00 • nonmember REVIEW-SHIPPING-E02	10.00 / 100.00	0.00 / 90.00
50.00 • nonmember REVIEW-SHIPPING-E01	10.00 / 60.00	8.00 / 58.00
150.00 • member REVIEW-SHIPPING-E04	0.00 / 100.00	0.00 / 100.00

Saved run 2026-10-19 • Candidate supplied review version
Recorded classroom review example
Saved macOS run with JDK 27 / release 21; current version and classroom results still need checking.

Trace the value used for shipping eligibility. Explain what a passing example still leaves unchecked.

© 2026 Inspire | Confidential

Your Revised Comment

1. Point to the changed location and the affected agreement.
2. State your evidence, or the question that still needs a run.
3. Give a concrete next action. Revise your own comment.

3 minutes to revise, then 2 minutes to connect the ideas. Next, we make confirmed examples repeatable checks.

© 2026 Inspire | Confidential

A Test Repeats a Known Check

A confirmed example gives us an input and an expected result.

Test code	Prepares that input, calls the behavior and compares the result.
JUnit	Finds and runs test methods, then reports what happened.

Today we translate supplied examples. Day 3 introduces selecting additional cases from rules.

© 2026 Inspire | Confidential

Arrange, Act, Assert

checkpoint/discount-and-shipping · src/test/java/training/quote/BasicsTest.java:5–10

```
5       @Test void emptyOrderHasNoCharge() {
6           for (boolean member : new boolean[]{false, true}) {
7               var quote = new
  QuoteCalculator().calculate(Money.amount("0.00"), member);
8               assertEquals(Money.amount("0.00"), quote.total());
9           }
10      }
```

Where do you see the input, the call and the comparison?

© 2026 Inspire | Confidential

Each Test Prepares Its Own State

You should be able to read one test and identify its starting state.

- Create the object and input that this test needs.
- Let the test run without another test running first.
- Take expected values from the confirmed example.

Use the project’s string-based Money.amount values. BigDecimal.equals also compares scale, so keep the project’s amount representation consistent.

© 2026 Inspire | Confidential

The Assertion Has a Defined Scope

checkpoint/discount-and-shipping · src/test/java/training/quote/BasicsTest.java:11–14

```
11      @Test void memberPriceRoundsOnce() {
12          var quote = new
    QuoteCalculator().calculate(Money.amount("100.01"), true);
13          assertEquals(Money.amount("80.01"), quote.memberPrice());
14      }
```

Which field does this test check? Where does its expected value come from?

© 2026 Inspire | Confidential

The Report Tells Us What Ran

Test summary	Tests run, Failures, Errors and Skipped.
Method names	Check the XML testcase entries against your command and source.
Failure stage	Separate compilation, setup, test discovery and assertion comparison.

Open target/surefire-reports/training.quote.BasicsTest.txt and the corresponding TEST-training.quote.BasicsTest.xml.

© 2026 Inspire | Confidential

Keep the Result with the Tested Code

1. Save the current report and relevant console output outside target/.
2. Record the command and the tested code version.
3. Keep the source of any uncommitted test and label it as uncommitted.

A later run can overwrite reports. A compilation failure can leave an older report behind. Save the current failure instead of reusing an old pass.

© 2026 Inspire | Confidential

What Can This Green Test Tell Us?

Read displayedTotalMatchesCalculation in BasicsTest.

Identify both sides of each assertion. Which behavior does it check? Which conclusion still needs another check?

2 minutes to read, 3 minutes for a selected explanation and feedback.

© 2026 Inspire | Confidential

A Relationship Check Protects That Relationship

checkpoint/discount-and-shipping • src/test/java/training/quote/BasicsTest.java:15–19

```
15      @Test void displayedTotalMatchesCalculation() {
16          var quote = new QuoteCalculator().calculate(Money.amount("50.00"),
    false);
17          assertEquals(quote.goods().add(quote.shipping()), quote.total());
18          assertTrue(new QuotePrinter().format(quote).contains("total=" +
    quote.total() + "\n"));
19      }
```

Compare all three BasicsTest methods. Explain their actual assertions before counting any supplied example as covered.

© 2026 Inspire | Confidential

The Prepared Project Has an Identified Starting Point

```
./mvnw -Dtest=BasicsTest test
git diff <test-start-sha> -- src/main
git status --short
```

Confirm the independent testing copy and its recorded starting SHA.

The trainer first runs BasicsTest. Keep this completed baseline separate from the earlier review candidate. Ask the coach to inspect any production changes.

© 2026 Inspire | Confidential

One Confirmed Example Becomes a Test

Given	Original price **110.00**		Member **No**	
When	Calculate the quote.			
Then Expected result	Original price **110.00**	Member price **110.00**	Promotion **20.00**	Goods payable **90.00**
	Shipping fee **10.00**	Total payable **100.00**		

Observation: Not recorded

Source example: US-02-E02

Watch the trainer’ s separate test copy. Your project still starts with BasicsTest.

© 2026 Inspire | Confidential

The Test Names Its Input and Call

Guide demonstration • QuoteExamplesTest.java:1–8

```
1 package training.quote;
2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.*;
4
5 class QuoteExamplesTest {
6     // US-02-E02: expected values come from the confirmed example.
7     @Test void shippingUsesGoodsAfterPromotion() {
8         var quote = new
  QuoteCalculator().calculate(Money.amount("110.00"), false);
```

The example determines the input. JUnit finds the method through @Test.

© 2026 Inspire | Confidential

Each Expected Amount Has a Source

Guide demonstration • QuoteExamplesTest.java:9–14

```
 9        assertEquals(Money.amount("110.00"), quote.original());
10        assertEquals(Money.amount("110.00"), quote.memberPrice());
11        assertEquals(Money.amount("20.00"), quote.promotion());
12        assertEquals(Money.amount("90.00"), quote.goods());
13        assertEquals(Money.amount("10.00"), quote.shipping());
14        assertEquals(Money.amount("100.00"), quote.total());
```

Choose one expected value. Which field of the supplied example gives it?

© 2026 Inspire | Confidential

The Display Also Has Expected Values

Guide demonstration • QuoteExamplesTest.java:15–19

```
15          String text = new QuotePrinter().format(quote);
16          assertTrue(text.contains("shipping=10.00\n"));
17          assertTrue(text.contains("total=100.00\n"));
18      }
19  }
```

Check the field label, value and line boundary. Later tests use the same approach for the other required display fields.

© 2026 Inspire | Confidential

A Named Run, Then the Whole Suite

```
./mvnw "-Dtest=QuoteExamplesTest#shippingUsesGoodsAfterPromotion" test
./mvnw test
```

After each run, check and save its current report before starting the next run.

Run beside pom.xml. On Windows use mvnw.cmd. Inspect new untracked test files directly because ordinary git diff omits them.

© 2026 Inspire | Confidential

Does the Result Match This Test Attempt?

- Find the actual method name and test count in the current report.
- Compare the assertions with the supplied example.
- Inspect test changes and confirm that production files remain unchanged.

The clean trainer copy has 1 test in the named run and 4 in the suite. Your count follows the methods actually present in your project.

© 2026 Inspire | Confidential

The Example-to-Test Link Makes Coverage Traceable

US-02-E02	QuoteExamplesTest#shippingUsesGoodsAfterPromotion
Assertions	Six amount fields, shipping text and total text
Result	Record your actual run, code version and remaining gaps.

Keep the same mapping notes for practice and Homework.

After the demonstration, the full method body leaves the screen. You can still consult BasicsTest, the API signatures and syntax.

© 2026 Inspire | Confidential

Your Next Supplied Example

Given	Original price **125.00**		Member **Yes**	
When	Calculate the quote.			
Then Expected result	Original price **125.00**	Member price **100.00**	Promotion **20.00**	Goods payable **80.00**
	Shipping fee **10.00**	Total payable **90.00**		

Observation: Not recorded

Source example: US-02-E07

Use your own prepared project. The trainer’ s demonstration test is separate from your work.

© 2026 Inspire | Confidential

Your Test, Your Decisions

1. Open or create QuoteExamplesTest.java in the existing test package.
2. Choose a method name, prepare the input and call calculate.
3. Write assertions whose expected values come from the supplied example.

20 minutes including the brief. Consult existing syntax and API signatures. Keep production behavior unchanged.

© 2026 Inspire | Confidential

Explain the Judgment Before Asking for Code

Input and result	What does your example give you? What must your test observe?
Location and syntax	Find the existing tests and calculate signature. Ask about imports or syntax.
Assertion	Point to the source of the expected value before writing it.

If you need a reference method to continue, record that help with your attempt.

© 2026 Inspire | Confidential

Which Assertion Would Reveal a Shipping Error?

You have checked the total.

Which other results does the example specify? Which assertion would tell you that a particular field was wrong?

Keep expected values tied to the example. When finished early, check your assertions and mapping.

© 2026 Inspire | Confidential

Your Actual Test Run

```
./mvnw "-Dtest=QuoteExamplesTest#<actual-method-name>" test
```

Green	Identify the executed method and the source of every expected value.
Red or blocked	Save the output and identify the stage where execution stopped.

Replace the method placeholder. Keep the expected result and product unchanged while investigating a failure.

© 2026 Inspire | Confidential

Save This Attempt for Feedback

- Save the current report, relevant console output and matching test source.
- Add the actual method and result to your existing example mapping.
- Keep expected/actual differences and environment blockers explicit.

10 minutes for running, checking and saving. We will use two real attempts for feedback next.

© 2026 Inspire | Confidential

What Can We Learn from Two Real Attempts?

Follow the input, call, expected values and actual result.

Which concrete assertion would you keep or improve? Give its source.

© 2026 Inspire | Confidential

Two Authors Explain Their Checks

Author • 90 seconds	Show your real test. Explain the input, call, expected-value source and run result.
Listeners • 30 seconds	Find the same issue or check in your own test.
Feedback • 1 minute	Connect one concrete assertion to the supplied example and current result.

Two selected attempts, 3 minutes each, inside the 8-minute sharing step.

© 2026 Inspire | Confidential

What If Both Component Amounts Are Wrong?

A relationship between calculated fields can still hold.

Does that also prove each amount matches the supplied example? Point to the assertions that answer this question.

Use an actual learner test. When both attempts are accurate, explain their sources and checked scope.

© 2026 Inspire | Confidential

Apply the Feedback to Your Own Test

1. Find a specific place to improve, or explain why it should stay.
2. Make any needed correction and run your own test again.
3. Update your mapping, actual result and unfinished items.

7 minutes: about 5 for checking and running, then 2 for your record. The displayed author’ s run is their result.

© 2026 Inspire | Confidential

Another Example, Less Guidance

Given	Original price **120.00**		Member **No**	
When	Calculate the quote.			
Then Expected result	Original price **120.00**	Member price **120.00**	Promotion **20.00**	Goods payable **100.00**
	Shipping fee **0.00**	Total payable **100.00**		

Observation: Not recorded

Source example: US-02-E04

Continue your project and mapping. Use this supplied input and expected result.

© 2026 Inspire | Confidential

Plan the Test Yourself

1. Decide which results you need to observe.
2. Prepare the input, call the behavior and write the assertions.
3. Ask for syntax or environment help after explaining your own judgment.

15 minutes including reopening your saved work. Keep to the supplied example and the existing product behavior.

© 2026 Inspire | Confidential

The Same Total Can Hide Different Amounts

Confirmed example	Original	Member?	Promotion	Goods	Shipping	Total
US-02-E02	110.00	No	20.00	90.00	10.00	100.00
US-02-E04	120.00	No	20.00	100.00	0.00	100.00

Which behavior does each example check? Which field assertion distinguishes them?

© 2026 Inspire | Confidential

The Suite and the Saved Version

```
./mvnw test
git status --short
git diff <test-start-sha> -- src/main
```

Save the current report and relevant console output outside the repository before another run.

Use the recorded starting SHA. Production differences should be empty, with no new production files in status.

© 2026 Inspire | Confidential

Save the Tests You Actually Checked

```
git add src/test/java/training/quote/QuoteExamplesTest.java
git diff --cached
git commit -m "test: check supplied quote examples"
git rev-parse HEAD
```

Check and stage BasicsTest separately if you changed it. Keep generated reports out of the commit. Preserve any remaining source changes with the run record.

© 2026 Inspire | Confidential

Your Homework Starts from This Saved Work

Code	Find its location, saved version and any uncommitted changes.
Mapping	Link each completed example to your actual method and assertions.
Result	Keep the real suite output, accepted help and unfinished checks.

A red result can be saved and explained. Keep your own attempt so you can continue it tonight.

© 2026 Inspire | Confidential